Lab-6-Assignment

AP25110010807

S JIGNESH

Q1.Code

main.cpp

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

struct Pair {
    int min;
    int max;
};

Pair getMinMax(const std::vector<int>& arr, int low, int high) {
    Pair result, left, right;

    if (low == high) {
        result.min = arr[low];
        result.max = arr[low];
        return result;
    }

    if (high == low + 1) {
        if (arr[low] > arr[high]) {
            result.max = arr[low];
            result.min = arr[high];
        } else {
            result.max = arr[high];
            result.min = arr[low];
        }
        return result;
    }

    int mid = low + (high - low) / 2;
    left = getMinMax(arr, low, mid);
    right = getMinMax(arr, mid + 1, high);

    result.min = std::min(left.min, right.min);
    result.max = std::max(left.max, right.max);

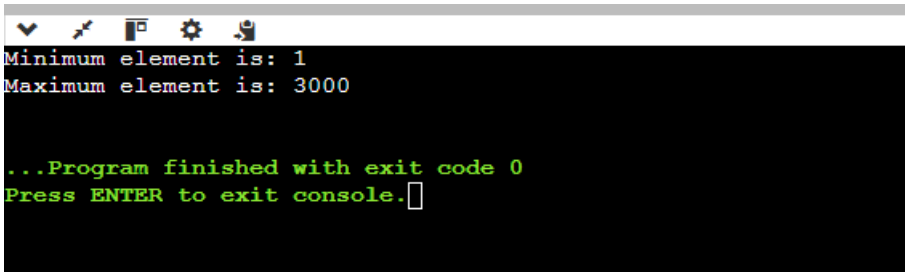
    return result;
}

int main() {
    std::vector<int> arr = {1000, 11, 445, 1, 330, 3000};
    Pair minmax = getMinMax(arr, 0, arr.size() - 1);

    std::cout << "Minimum element is: " << minmax.min << std::endl;
    std::cout << "Maximum element is: " << minmax.max << std::endl;

    return 0;
}
```

Q1.Output:



Q1.Analysis and comparison :

1.Traditional approach takes $2(n-1)$ comparisons in the worst case, but divide and conquer reduces it to $\frac{3n}{2}-2$ comparisons by comparing pairs first.

2.Both methods have a time complexity of $O(n)$, but divide and conquer saves around 25% on total comparison operations.

3.Linear search takes $O(1)$ auxiliary space, whereas divide and conquer uses $O(\log n)$ extra space because of recursive stack calls.

4.Divide and conquer breaks the array into independent subproblems, which makes it easier to run in parallel on multi-core processors compared to sequential search.

Q2.Code

main.cpp

```cpp
#include <iostream>
#include <vector>
#include <climits>
#include <algorithm>

int maxCrossingSum(const std::vector<int>& arr, int low, int mid, int high) {
    int sum = 0;
    int left_sum = INT_MIN;
    for (int i = mid; i >= low; i--) {
        sum += arr[i];
        if (sum > left_sum) left_sum = sum;
    }

    sum = 0;
    int right_sum = INT_MIN;
    for (int i = mid + 1; i <= high; i++) {
        sum += arr[i];
        if (sum > right_sum) right_sum = sum;
    }

    return left_sum + right_sum;
}

int maxSubArraySum(const std::vector<int>& arr, int low, int high) {
    if (low == high) return arr[low];

    int mid = low + (high - low) / 2;

    int left_max = maxSubArraySum(arr, low, mid);
    int right_max = maxSubArraySum(arr, mid + 1, high);
    int cross_max = maxCrossingSum(arr, low, mid, high);

    return std::max({left_max, right_max, cross_max});
}

int main() {
    std::vector<int> arr = {-2, -3, 4, -1, -2, 1, 5, -3};
    int max_sum = maxSubArraySum(arr, 0, arr.size() - 1);

    std::cout << "Maximum contiguous sum is: " << max_sum << std::endl;

    return 0;
}
```

Q2.Output

```
Maximum contiguous sum is: 7


...Program finished with exit code 0
Press ENTER to exit console.
```

Q3.Code

main.cpp

```cpp
#include <iostream>

void strassen2x2(int A[2][2], int B[2][2], int C[2][2]) {
    int P1 = A[0][0] * (B[0][1] - B[1][1]);
    int P2 = (A[0][0] + A[0][1]) * B[1][1];
    int P3 = (A[1][0] + A[1][1]) * B[0][0];
    int P4 = A[1][1] * (B[1][0] - B[0][0]);
    int P5 = (A[0][0] + A[1][1]) * (B[0][0] + B[1][1]);
    int P6 = (A[0][1] - A[1][1]) * (B[1][0] + B[1][1]);
    int P7 = (A[0][0] - A[1][0]) * (B[0][0] + B[0][1]);

    C[0][0] = P5 + P4 - P2 + P6;
    C[0][1] = P1 + P2;
    C[1][0] = P3 + P4;
    C[1][1] = P1 + P5 - P3 - P7;
}

int main() {
    int A[2][2] = {{1, 2}, {3, 4}};
    int B[2][2] = {{5, 6}, {7, 8}};
    int C[2][2];

    strassen2x2(A, B, C);

    std::cout << "Resulting Matrix C:" << std::endl;
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            std::cout << C[i][j] << " ";
        }
        std::cout << std::endl;
    }

    return 0;
}
```

Q3.Output

```
Resulting Matrix C:
19 22
43 50


...Program finished with exit code 0
Press ENTER to exit console.
```

1. Normal matrix multiplication needs 8 multiplications, while Strassen's reduces it to 7 by using clever matrix additions and subtractions.

2. Standard multiplication has $O(n^3)$ time complexity, but Strassen's improves it to O(n^log2(7)) is approximately equal to O(n^2.81)

3. Fewer multiplications make Strassen's algorithm much faster when dealing with large size matrices.

4. Strassen's uses extra memory for storing sub-matrices, so normal multiplication is better for small inputs

Q4.Code

main.cpp

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <iomanip>

struct Item {
    int value;
    int weight;
};

bool compare(Item a, Item b) {
    double r1 = (double)a.value / a.weight;
    double r2 = (double)b.value / b.weight;
    return r1 > r2;
}

double fractionalKnapsack(int W, std::vector<Item>& items) {
    std::sort(items.begin(), items.end(), compare);

    double totalValue = 0.0;

    for (const auto& item : items) {
        if (W >= item.weight) {
            W -= item.weight;
            totalValue += item.value;
        } else {
            totalValue += item.value * ((double)W / item.weight);
            break;
        }
    }

    return totalValue;
}

int main() {
    int W = 50;
    std::vector<Item> items = {{60, 10}, {100, 20}, {120, 30}};

    double max_value = fractionalKnapsack(W, items);

    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Maximum value in Knapsack = " << max_value << std::endl;

    return 0;
}
```

Q4.Output

```
Maximum value in Knapsack = 240.00


...Program finished with exit code 0
Press ENTER to exit console.
```